NATURAL LANGUAGE PROCESSING

Natural Language Processing: Foundations, Preprocessing, and Vector Semantics

A complete first-principles study companion covering linguistic structure, text cleaning, evaluation math, and dense word embeddings

FROM FIRST PRINCIPLES · EVERY EQUATION AND NUMERICAL CALCULATION STEPPED THROUGH IN FULL

How to use this companion. This study guide explains the core foundations of natural language processing step by step. We assume this is your very first time meeting this topic. Every idea begins with an everyday analogy before any math appears. Whenever a formula is introduced, we name every single symbol. We write down every algebraic step from one line to the next. We explain in one plain sentence why each step is taken. All numerical calculations are worked out in full without skipping numbers. The callout boxes are marked by clear uppercase labels: **THE INTUITION** states the core mechanism in simple words. **EVERYDAY PICTURE** connects the idea to familiar daily life. **WORKED EXAMPLE** walks through full calculations on a light grey background. **WATCH OUT** highlights common traps and mistakes. **WAIT — REALLY?** points out honest, surprising truths. **KEY TAKEAWAY** summarizes the main rule to keep in mind.

1 The Mystery of Language: Rules, Meaning, and Ambiguity

Computers require rigid formal rules, while human speech bends syntax and depends on shared context.

Computers run on formal code like Python or SQL. In code, every single character follows strict grammar rules. If you omit one bracket, the entire compiler halts. Human language behaves very differently. Linguists call human languages like English, Hindi, and Malayalam **natural languages**. They grew naturally across generations of human interaction. They change continually, carry exceptions, and depend on shared cultural understanding.

> **EVERYDAY PICTURE**
>
> Think of ordering breakfast at a cafe counter. You look at the server and ask: "Can you pass the salt?" A literal computer interprets this as an inquiry about your physical

arm strength. The human server understands your intent and hands you the shaker. Natural communication packs hidden layers of social intention that never appear in a dictionary definition.

THE INTUITION

Natural Language Processing (NLP) builds computational systems to analyze, understand, and generate human speech. NLP divides into two primary directions: **Natural Language Understanding** (NLU) maps human sentences into structured computer concepts. **Natural Language Generation** (NLG) maps computer data back into fluent human sentences.

1.1 Position in Computer Science and Brief History

Where does NLP fit within computing? NLP sits at the crossroads of Artificial Intelligence, Computer Science, and Linguistics. It connects directly to Information Retrieval, Machine Translation, and Machine Learning. Historians trace NLP across four major eras:

1. **1950s–1980s (Symbolic Era)**: Hand-crafted grammatical rules and dictionary lookups.
2. **1990s–2000s (Statistical Era)**: Probabilistic counts and N-gram language models.
3. **2010s (Neural Era)**: Dense word vectors, recurrent neural nets, and word2vec.
4. **2020s (Transformer and LLM Era)**: Large language models and prompt engineering.

1.2 The Six Levels of Linguistic Analysis

To parse language, computational models inspect text across six distinct levels:

1. **Phonetics and Orthography**: The physical sounds we speak and the written spelling characters.
2. **Morphology**: The internal structure of words formed by basic units called **morphemes**. For instance, "unhappiness" contains three morphemes: "un-" (prefix meaning not), "happy" (root stem), and "-ness" (noun suffix).
3. **Syntax**: The grammatical arrangement of words into valid hierarchical sentences.
4. **Semantics**: The literal meaning of individual words and how they combine into propositions.
5. **Pragmatics**: How real-world context and speaker intent change literal meaning into social action.
6. **Discourse**: How meaning links across multiple consecutive sentences in a story or dialogue.

1.3 The Challenge of Ambiguity

Why is NLP hard? The main reason is **ambiguity**. A single surface string can trigger multiple competing interpretations:

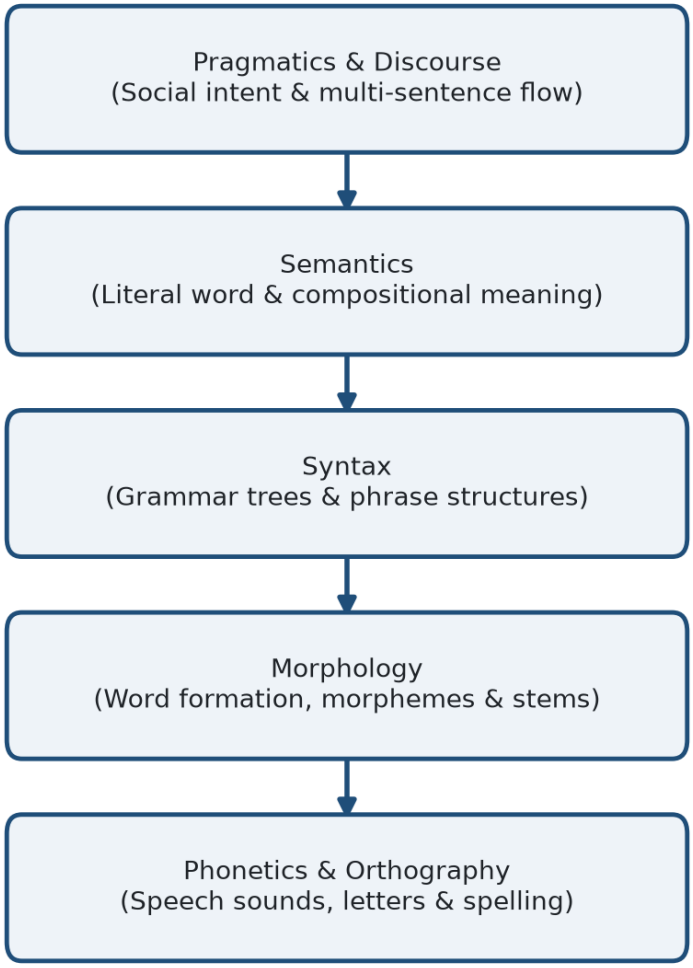


Figure 1. The six levels of linguistic analysis. Each level feeds information upward to build complete understanding.

- **Lexical Ambiguity**: One word has multiple distinct meanings. In "He went to the bank", the word "bank" could mean a financial building or a muddy river edge.
- **Syntactic Ambiguity**: A sentence allows multiple valid grammatical parse trees. In "I saw the man with a telescope", does the telescope belong to the observer or to the man?
- **Semantic Ambiguity**: Words combine into multiple literal readings. In "Flies like rice", are insects enjoying rice, or does someone toss grain quickly?
- **Pragmatic Ambiguity**: Context dictates whether a phrase is literal, sarcastic, or a request.

WORKED EXAMPLE · *Resolving syntactic attachment ambiguity with grammar trees*

Consider the classic sentence: "I saw the man with a telescope." Unpack both structural parse trees and explain why a parser needs real-world context.

Step 1 **Identify the movable prepositional phrase.** The phrase is "with a telescope".

Step 2 **Test Verb Attachment (Instrument reading).** The phrase attaches directly

to the verb phrase "saw". The telescope functions as the instrument of perception. Meaning: "Using a telescope, I observed a man."

Step 3 **Test Noun Phrase Attachment (Attribute reading).** The phrase attaches directly to the noun phrase "the man". The telescope serves as an attribute or possession of that man. Meaning: "I saw a man who held a telescope."

Step 4 **State the conclusion.** Both readings follow correct English syntax rules. A computer cannot resolve this choice without outside world knowledge.

WATCH OUT

Keep polysemy distinct from homonymy. **Polysemy** occurs when a single word holds related senses (like "paper" as a physical sheet or an academic article). **Homonymy** occurs when two completely unrelated words share spelling by historical coincidence (like financial "bank" and river "bank").

KEY TAKEAWAY

Human language operates on six layered levels from sounds to discourse. Ambiguity at every layer means algorithms must rely on surrounding context.

Where it's used. Voice assistants decode sound with phonetics. They parse syntax, extract semantics, and infer your intent through pragmatics.

2 The Text Preprocessing Pipeline: Cleaning and Normalizing

Raw text contains noise; a four-step pipeline converts dirty strings into clean tokens.

Computer models cannot read raw, unformatted text files directly. Real text contains messy capitalization, odd punctuation marks, emojis, and slang. We process this text through a systematic pipeline before running machine learning algorithms.



Figure 2. The text preprocessing pipeline. It transforms raw messy text into clean, normalized tokens.

The preprocessing pipeline follows four core stages:

1. **Text Cleaning**: Converting letters to lowercase and removing special characters via regular expressions.
2. **Tokenization**: Splitting continuous sentences into individual words or punctuation

marks.

3. **Stop-Word Removal**: Dropping frequent grammatical words like "the", "is", and "at".
4. **Stemming or Lemmatization**: Normalizing words down to their common dictionary root.

EVERYDAY PICTURE

Imagine harvesting vegetables from an earthy garden plot. First, you wash off the loose soil (text cleaning). Next, you chop the carrots into bite-sized pieces (tokenization). Then, you pick out the tough stems and leaves (stop-word removal). Finally, you peel them down to the edible core (lemmatization).

THE INTUITION

Stemming uses fast heuristic rules to chop off prefixes and suffixes. It acts like a fast kitchen knife. It often leaves non-words like "studi" for "studying". **Lemmatization** consults a full vocabulary database like WordNet. It inspects the word's part-of-speech role and returns a genuine dictionary headword, called a **lemma**.

WORKED EXAMPLE · *Walking through the complete preprocessing pipeline*

Take this raw message string: `"The 3 cats were happily playing, wasn't it great?!"` Show the exact result after each stage of the pipeline.

Step 1 **Clean characters and lowercase letters.** Remove punctuation marks and lowercase letters: `"the 3 cats were happily playing wasnt it great"`

Step 2 **Tokenize text into a word list.** Split by spaces into separate tokens: `['the', '3', 'cats', 'were', 'happily', 'playing', 'wasnt', 'it', 'great']`

Step 3 **Filter out standard English stop-words.** Drop common words (`'the'`, `'were'`, `'it'`): `['3', 'cats', 'happily', 'playing', 'wasnt', 'great']`

Step 4 **Apply Porter stemming rules.** Chop word endings using heuristic rules: `'cats'` → `'cat'`, `'happily'` → `'happili'`, `'playing'` → `'play'`.

Step 5 **Apply WordNet lemmatization with verb tags.** Lookup genuine vocabulary headwords: `'cats'` → `'cat'`, `'playing'` (tagged as verb) → `'play'`. Notice `'happily'` remains `'happily'` because it is a valid dictionary adverb.

WATCH OUT

Do not lowercase letters blindly when case carries key meaning. Converting "US" (the United States) into "us" (the pronoun) destroys geographic information. Decide if your task requires case sensitivity before running lowercase steps.

KEY TAKEAWAY

Preprocessing shrinks total vocabulary size. Stemming strips endings fast by rules, while lemmatization checks dictionary tags to return valid lemmas.

Where it's used. Search engines like Google preprocess billions of web pages. Lemmatization ensures a search for "running shoe" matches pages describing "run shoes".

3 Measuring Success: The Confusion Matrix and Metric Math

Accuracy fails when classes are imbalanced; precision and recall reveal the truth.

When an NLP classifier labels text messages as spam or legitimate mail (ham), how do we measure performance? We evaluate predictions on a test set and tally counts in a 2×2 **confusion matrix**.

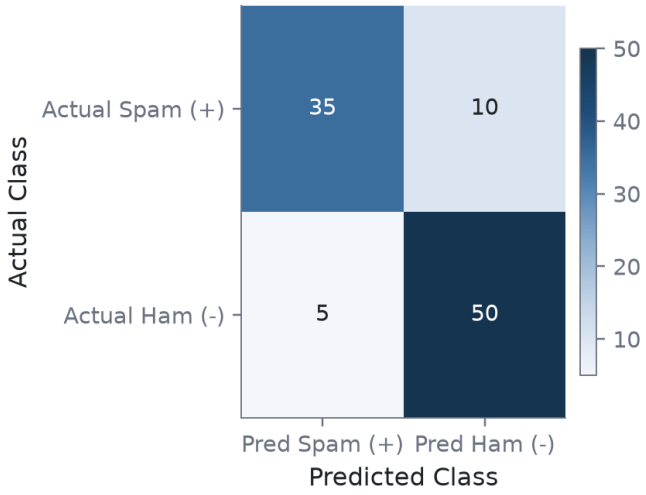


Figure 3. A 2×2 confusion matrix. It records true positives, false alarms, missed targets, and true negatives.

Let us define the four possible outcomes:

- **True Positive (TP)**: Spam correctly identified as spam.
- **False Positive (FP)**: Normal email wrongly flagged as spam (a false alarm).
- **False Negative (FN)**: Spam wrongly left in the regular inbox (a missed detection).
- **True Negative (TN)**: Normal email correctly left in the inbox.

Now we build every metric formula step by step, naming each symbol:

3.1 Accuracy

Accuracy measures the proportion of all decisions that were correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

Where:

- $TP + TN$ is the total count of correct classifications.
- $TP + TN + FP + FN$ is the grand total count of all evaluated messages.

3.2 Precision

Precision measures reliability when the model sounds an alarm. Out of everything labeled positive, what fraction was truly positive?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

Where:

- TP is the count of correctly caught positive items.
- $TP + FP$ is the total count of items the model predicted as positive.

3.3 Recall (Sensitivity)

Recall measures the model's coverage. Out of all genuine positive cases in the dataset, what fraction did the model find?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

Where:

- TP is the count of caught positive items.
- $TP + FN$ is the total count of actual positive items in reality.

3.4 Specificity (True Negative Rate)

Specificity measures how well the model avoids false alarms on negative data:

$$\text{Specificity} = \frac{TN}{TN + FP} \tag{4}$$

Where:

- TN is the count of correctly identified negative instances.
- $TN + FP$ is the total count of actual negative instances.

3.5 The F1-Score

Precision and recall often pull in opposite directions. If a classifier labels everything positive, recall hits 100% but precision drops. The **F1-score** computes their **harmonic mean**:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

> **THE INTUITION**
>
> Why do we choose a harmonic mean instead of a simple average? A regular arithmetic average allows a terrible score to hide behind a great score. If precision is 1.0 and recall

is 0.0, their arithmetic mean is 0.5. The harmonic mean punishes the zero and falls straight to 0.0. It requires both numbers to be strong to earn a high score.

WORKED EXAMPLE · *Step-by-step arithmetic evaluation of a spam classifier*

Suppose a test set contains 100 messages. The model produces: $TP = 35$, $FP = 10$, $FN = 5$, and $TN = 50$. Compute all five evaluation metrics with complete arithmetic steps.

Step 1 **Check the total count.** $TP + FP + FN + TN = 35 + 10 + 5 + 50 = 100$. The numbers account for every message in the sample.

Step 2 **Compute Accuracy.** Add true positives and true negatives: $35 + 50 = 85$. Divide by the total: $\frac{85}{100} = 0.85$. Accuracy is **85.0%**.

Step 3 **Compute Precision.** Add true positives and false alarms: $35 + 10 = 45$. Divide TP by this sum: $\frac{35}{45} \approx 0.7778$. Precision is **77.78%**.

Step 4 **Compute Recall.** Add true positives and missed cases: $35 + 5 = 40$. Divide TP by this sum: $\frac{35}{40} = 0.8750$. Recall is **87.50%**.

Step 5 **Compute Specificity.** Add true negatives and false alarms: $50 + 10 = 60$. Divide TN by this sum: $\frac{50}{60} \approx 0.8333$. Specificity is **83.33%**.

Step 6 **Compute F1-Score.** Multiply precision by recall: $0.7778 \times 0.8750 = 0.680575$. Add precision and recall: $0.7778 + 0.8750 = 1.6528$. Divide the product by the sum: $\frac{0.680575}{1.6528} \approx 0.41177$. Multiply by 2: $2 \times 0.41177 \approx 0.8235$. The final F1-score is **0.8235** (or **82.35%**).

WAIT — REALLY?

In spam filtering, a false positive hurts far more than a false negative! A false negative leaves an annoying advertisement in your inbox. A false positive throws an urgent job interview offer into the spam folder. That is why engineers configure spam filters for high precision over recall.

KEY TAKEAWAY

Never rely on accuracy alone when classes are unbalanced. Check precision for false alarm safety, recall for coverage, and F1-score for harmony.

Where it's used. Medical diagnosis models require high recall so they never miss an illness. Legal discovery systems balance precision to avoid sorting through millions of irrelevant pages.

4 Meaning as a Neighborhood: The Distributional Idea

You shall know a word by the company it keeps; context reveals meaning.

How should a computer represent the meaning of a word? In older databases, words were unique integer IDs: "cat" was ID 412, and "dog" was ID 890. These IDs told the computer nothing about word relationships. To the program, "dog" was as mathematically distant from "puppy" as from "refrigerator".

> **EVERYDAY PICTURE**
>
> Imagine reading a travel article about a dish containing an unfamiliar ingredient named "ongchoi". You read: "Stir-fried ongchoi with garlic is delicious." "Fresh green ongchoi leaves taste crisp in soup." Even without a dictionary, you know ongchoi is an edible leafy green vegetable. Its surrounding company revealed its meaning.

> **THE INTUITION**
>
> Linguist J.R. Firth stated in 1957: *"You shall know a word by the company it keeps."* This principle is called the **distributional hypothesis**. Words that appear in similar textual environments share similar semantic meanings. If "oculist" and "eye-doctor" appear near "glasses", "vision", and "examined", they sit close together in semantic space.

Linguists organize word relations into structured families:

- **Synonymy**: Words sharing an identical sense (e.g., "sofa" and "couch").
- **Antonymy**: Words with opposite values on one semantic dimension (e.g., "hot" and "cold").
- **Similarity**: Words sharing core physical or conceptual features (e.g., "behave" and "obey").
- **Relatedness**: Words connected by situations or semantic fields (e.g., "coffee", "cup", "waiter").
- **Affective Dimensions**: Words carry emotional weight across three axes: **valence** (pleasantness), **arousal** (intensity), and **dominance** (control).

> **KEY TAKEAWAY**
>
> *Do not treat words as isolated symbol IDs. Words acquire meaning through their neighbors across a large collection of text.*

Where it's used. Search engines use semantic similarity to match queries like "cheap flights" to documents offering "budget airline tickets".

5 Counting Words: Co-occurrence, Angle Similarity, and TF-IDF

Raw counts favour common words; angle similarity and TF-IDF fix the bias.

To turn the distributional hypothesis into math, we tally word counts in a table. We call this a **co-occurrence matrix**. Rows represent target words. Columns represent context words

appearing within a sliding window C (such as ± 2 words).

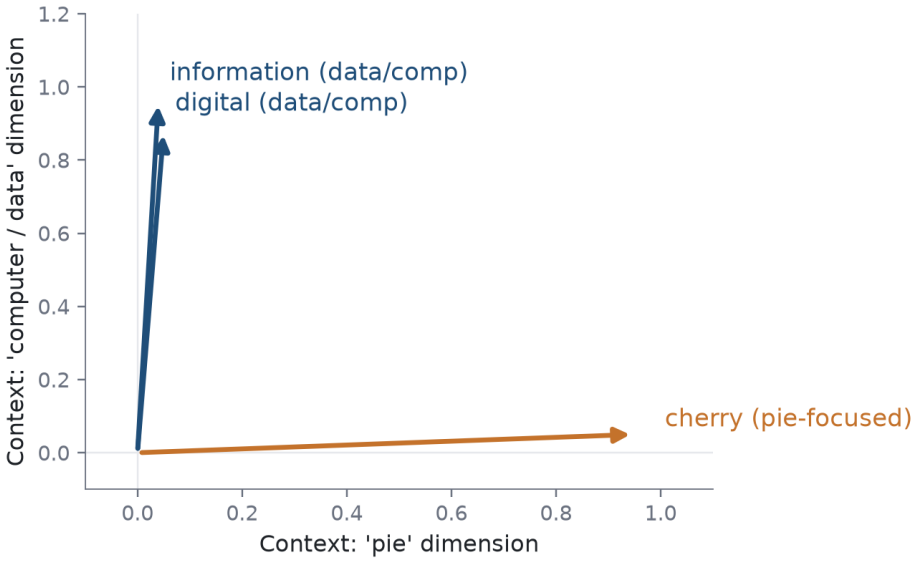


Figure 4. Geometric vectors for three words. The small angle between 'digital' and 'information' shows their high similarity.

Consider this count table drawn from textbook corpus experiments:

Word	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

5.1 The Dot Product and Its Length Flaw

We could compare two vectors $\mathbf{u}$ and $\mathbf{v}$ using their **dot product**:

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{d} u_i v_i \tag{6}$$

Where:

- $\mathbf{u}$ and $\mathbf{v}$ are vectors of length d.
- u_i and v_i are the numerical counts in dimension i.

Why is the raw dot product flawed? It suffers from a severe **length bias**. Frequent words like "the", "have", or "good" occur thousands of times across texts. Their long vector lengths artificially inflate dot products, making them look similar to everything.

5.2 Cosine Similarity: Measuring the Pure Angle

We eliminate length bias by dividing the dot product by the **Euclidean length** (L_2 norm) of both vectors:

$$\text{cosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{7}$$

Where:

- $\|\mathbf{u}\| = \sqrt{\sum_{i=1}^{d} u_i^2}$ is the geometric length of vector $\mathbf{u}$.
- $\|\mathbf{v}\| = \sqrt{\sum_{i=1}^{d} v_i^2}$ is the geometric length of vector $\mathbf{v}$.

Because word counts are non-negative, this score ranges strictly from 0.0 (orthogonal, sharing no context) to 1.0 (pointing in the identical direction).

WORKED EXAMPLE · *Step-by-step cosine similarity calculation*

Calculate the exact cosine similarity between "cherry" and "information" using our count matrix.

Step 1 **State the vector components.** $\mathbf{u}$ = cherry = $[442, 8, 2]$. $\mathbf{v}$ = information = $[5, 3982, 3325]$.

Step 2 **Calculate the dot product $\mathbf{u} \cdot \mathbf{v}$.** Multiply matching coordinates: $442 \times 5 = 2210$. $8 \times 3982 = 31856$. $2 \times 3325 = 6650$. Sum all three products: $\mathbf{u} \cdot \mathbf{v} = 2210 + 31856 + 6650 = 40716$.

Step 3 **Calculate the squared length of vector u.** Square each component: $442^2 = 195364$. $8^2 = 64$. $2^2 = 4$. Sum: $195364 + 64 + 4 = 195432$. Take the square root: $\|\mathbf{u}\| = \sqrt{195432} \approx 442.0769$.

Step 4 **Calculate the squared length of vector v.** Square each component: $5^2 = 25$. $3982^2 = 15856324$. $3325^2 = 11055625$. Sum: $25 + 15856324 + 11055625 = 26911974$. Take the square root: $\|\mathbf{v}\| = \sqrt{26911974} \approx 5187.6752$.

Step 5 **Multiply the two vector lengths.** $\|\mathbf{u}\|\|\mathbf{v}\| = 442.0769 \times 5187.6752 \approx 2293351.4$.

Step 6 **Divide dot product by length product.** $\text{cosine}(\mathbf{u}, \mathbf{v}) = \frac{40716}{2293351.4} \approx 0.01775$.

Step 7 **Interpret the score.** The cosine similarity is tiny (**0.0178**). This confirms that "cherry" and "information" share almost no semantic context. By contrast, comparing "digital" and "information" yields a high cosine score of **0.9963**.

5.3 TF-IDF Weighting: Taming Frequent Terms

Raw counts have another issue: universally common words dominate frequency tables. We balance raw counts using **TF-IDF** (Term Frequency–Inverse Document Frequency).

Term Frequency (TF). We squash raw counts with a base-10 logarithm. A word appearing ten times should not outweigh another tenfold:

$$\text{TF}(t, d) = \log_{10}(\text{count}(t, d) + 1) \tag{8}$$

Inverse Document Frequency (IDF). If a term appears across every document, it holds no distinguishing power. Let N be the total documents in the library, and df_t be the count of documents containing term t:

$$\text{IDF}(t) = \log_{10}\left(\frac{N}{df_t}\right) \tag{9}$$

Combined TF-IDF Score. We multiply the two terms:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{10}$$

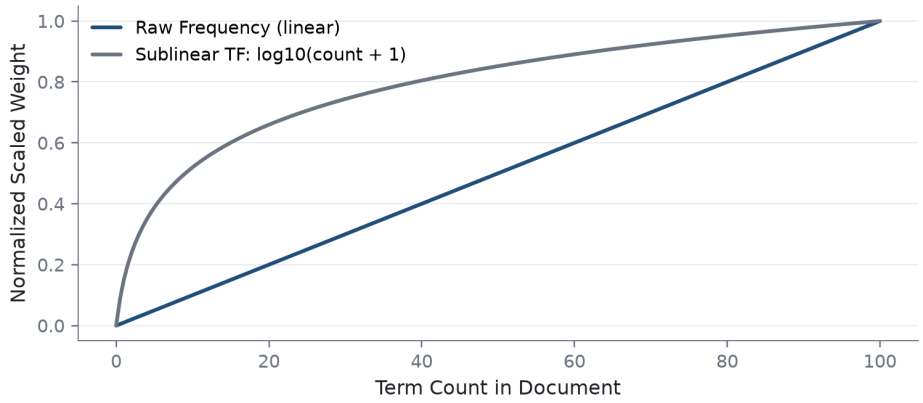


Figure 5. Sublinear term frequency scaling. Taking the log prevents words with huge raw counts from dominating.

WORKED EXAMPLE · *Step-by-step TF-IDF computation on Shakespeare plays*

Suppose Shakespeare wrote $N = 37$ plays. In a target play, the word "wit" appears count $= 20$ times. The word "wit" occurs in $df = 34$ of the 37 plays. Compute its complete TF-IDF score.

Step 1 **Compute squashed Term Frequency.** Add 1 to count: $20 + 1 = 21$. Take the base-10 log: $\text{TF} = \log_{10}(21) \approx 1.3222$.

Step 2 **Compute Inverse Document Frequency.** Compute document ratio: $\frac{N}{df} = \frac{37}{34} \approx 1.0882$. Take the base-10 log: $\text{IDF} = \log_{10}(1.0882) \approx 0.0367$.

Step 3 **Multiply both factors.** $\text{TF-IDF} = 1.3222 \times 0.0367 \approx 0.0485$.

Step 4 **Compare with a universal word.** If a word like "good" appears in all 37 plays ($df = 37$), its $\text{IDF} = \log_{10}(37/37) = \log_{10}(1) = 0$. Its TF-IDF drops to exactly **0.0**, removing it from consideration.

KEY TAKEAWAY

Cosine similarity removes length bias by dividing by geometric norms. TF-IDF down-weights words that appear everywhere, boosting rare and informative terms.

6 Dense Vectors: Why Word2Vec Shrinks the Space

Sparse 50,000-word tables waste memory; dense 300-number vectors capture latent meaning.

Co-occurrence tables produce vectors with $|V|$ dimensions, where $|V|$ is the total vocabulary size (often 20,000 to 50,000). These vectors are **sparse**, meaning almost every number is zero. Sparse vectors waste storage memory, fail to capture synonyms, and cannot generalize to unseen word pairs.

To fix this, modern NLP trains **dense embeddings**. Every word becomes a compact vector of 100 to 300 continuous numbers.

In 2013, Mikolov and colleagues introduced **Word2Vec** with two primary training setups:



Figure 6. The CBOW architecture. Surrounding context words average together to predict the center target word.

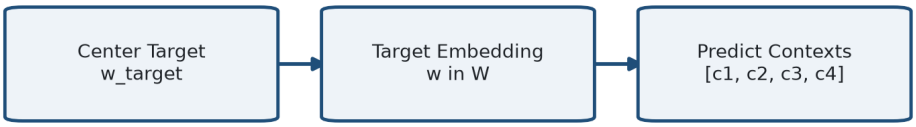


Figure 7. The Skip-gram architecture. A single target word predicts surrounding context words.

1. **Continuous Bag-of-Words (CBOW)**: Looks at surrounding context words within window C and predicts the missing center word. It trains quickly and works well for frequent words.
2. **Skip-Gram with Negative Sampling (SGNS)**: Looks at a single center word and predicts its surrounding context words. It performs exceptionally well on small corpora and rare words.

> **THE INTUITION**
>
> Window size controls what embeddings learn. A small window like $C = \pm 2$ captures grammatical and functional synonyms. A large window like $C = \pm 5$ or ± 10 captures broad topical associations like "doctor" and "hospital".

> **KEY TAKEAWAY**
>
> *Dense vectors compress vocabulary into low-dimensional semantic space. Skip-gram predicts context from a center word, while CBOW predicts the center word from context.*

7 Skip-Gram with Negative Sampling: The Math Step by Step

A binary classifier turns vector dot products into probabilities using logistic loss.

Now we unpack the complete mathematics of Skip-Gram with Negative Sampling (SGNS). Every single formula is stepped through line by line, naming every symbol.

7.1 Formulation as Binary Classification

Instead of computing expensive softmax probabilities over all 50,000 words, SGNS reframes training as simple binary logistic regression. Given a target word w and a context word c, the model answers: *Is word c a real neighbor of word w?*

Let:

- $\mathbf{w} \in \mathbb{R}^d$ be the target embedding vector of length d.
- $\mathbf{c} \in \mathbb{R}^d$ be the context embedding vector of length d.
- $\sigma(z) = \frac{1}{1+e^{-z}}$ be the **sigmoid function**, squashing any real number z into a probability in $(0, 1)$.

The probability that c is a genuine positive neighbor of w is:

$$P(+ \mid w, c) = \sigma(\mathbf{w} \cdot \mathbf{c}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{c}}} \tag{11}$$

The probability that c is not a neighbor (a negative sample) is:

$$P(- \mid w, c) = 1 - P(+ \mid w, c) = 1 - \sigma(\mathbf{w} \cdot \mathbf{c}) = \sigma(-\mathbf{w} \cdot \mathbf{c}) = \frac{1}{1 + e^{\mathbf{w} \cdot \mathbf{c}}} \tag{12}$$

7.2 The Loss Function for One Target Word

For each true co-occurring pair (w, c_{pos}), we pick k fake noise words $(c_{\text{neg},1}, \ldots, c_{\text{neg},k})$ sampled from the unigram noise distribution $P_n(w) \propto U(w)^{3/4}$. We want to maximize $P(+ \mid w, c_{\text{pos}})$ while maximizing $P(- \mid w, c_{\text{neg},i})$. Taking negative log-likelihood gives our **log loss**:

$$L = -\log \sigma(\mathbf{w} \cdot \mathbf{c}_{\text{pos}}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{w} \cdot \mathbf{c}_{\text{neg},i}) \tag{13}$$

7.3 Deriving the Gradients

We use the key calculus identity for the sigmoid function:

$$\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z) \tag{14}$$

and

$$\frac{d}{dz} \log \sigma(-z) = -\sigma(z) \tag{15}$$

Let us define target label t_j:

$$t_j = \begin{cases} 1 & \text{if } c_j \text{ is the positive context word } c_{\text{pos}} \\ 0 & \text{if } c_j \text{ is a negative noise word } c_{\text{neg}} \end{cases} \tag{16}$$

The prediction error for any context candidate c_j is:

$$e_j = \sigma(\mathbf{w} \cdot \mathbf{c}_j) - t_j \tag{17}$$

By the chain rule of calculus:

1. The derivative of loss L with respect to context vector $\mathbf{c}_j$ is:

$$\frac{\partial L}{\partial \mathbf{c}_j} = (\sigma(\mathbf{w} \cdot \mathbf{c}_j) - t_j)\mathbf{w} = e_j \mathbf{w} \tag{18}$$

Why: We take the derivative with respect to $\mathbf{c}_j$, so the dot product rule leaves vector $\mathbf{w}$ scaled by prediction error.

2. The derivative of loss L with respect to target vector $\mathbf{w}$ is:

$$\frac{\partial L}{\partial \mathbf{w}} = \sum_{j=1}^{1+k} (\sigma(\mathbf{w} \cdot \mathbf{c}_j) - t_j)\mathbf{c}_j = \sum_{j=1}^{1+k} e_j \mathbf{c}_j \tag{19}$$

Why: The target word $\mathbf{w}$ interacts with every context word, so its gradient sums errors across all $1 + k$ candidates.

7.4 Stochastic Gradient Descent (SGD) Update

With learning rate η, we adjust parameters in the opposite direction of the gradient:

$$\mathbf{c}_j \leftarrow \mathbf{c}_j - \eta \frac{\partial L}{\partial \mathbf{c}_j} = \mathbf{c}_j - \eta\, e_j \mathbf{w} \tag{20}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \eta \sum_{j=1}^{1+k} e_j \mathbf{c}_j \tag{21}$$

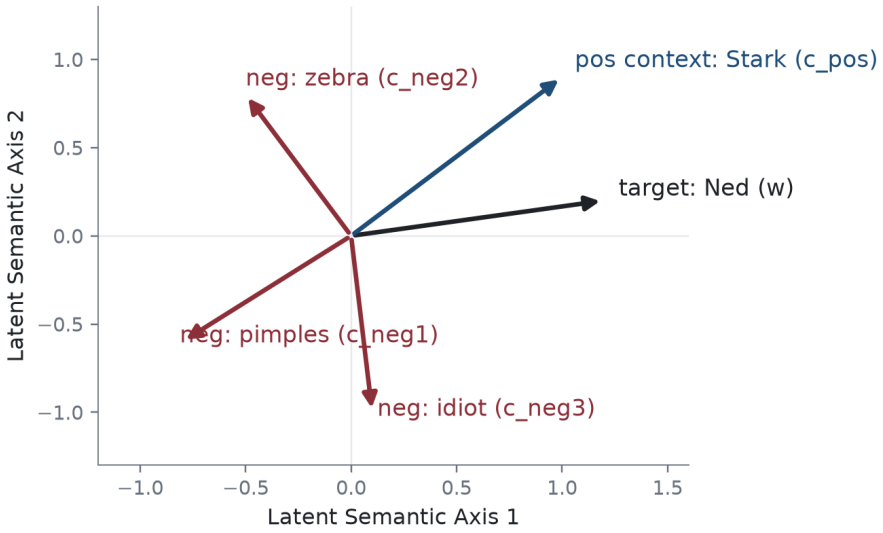


Figure 8. Geometric update under SGNS. The target vector shifts toward true context and moves away from negative samples.

WORKED EXAMPLE · *Full numerical walkthrough of SGNS on 'Ned Stark'*

Corpus sentence: `"Ned Stark is the most honourable man"`. Target word: $w =$ "Ned". Positive context word: $c_1 =$ "Stark" ($t_1 = 1$). $k = 3$ negative noise words: $c_2 =$ "pimples", $c_3 =$ "zebra", $c_4 =$ "idiot" ($t_2 = t_3 = t_4 = 0$). Learning rate: $\eta = 0.05$. Dimensionality: $d = 3$.

Initial embeddings:

$$\begin{aligned}\mathbf{w} &= [-0.018,\ 0.404,\ -0.317]\\ \mathbf{c}_1\ (\text{Stark}) &= [0.116,\ 0.723,\ -0.689]\\ \mathbf{c}_2\ (\text{pimples}) &= [-0.940,\ 0.601,\ 0.146]\\ \mathbf{c}_3\ (\text{zebra}) &= [-0.622,\ 0.811,\ 0.640]\\ \mathbf{c}_4\ (\text{idiot}) &= [-0.077,\ -0.375,\ -0.056]\end{aligned}$$

Step 1 **Forward Pass: Dot Products.** Calculate $\mathbf{w} \cdot \mathbf{c}_j$ for each word:

- Stark: $(-0.018)(0.116) + (0.404)(0.723) + (-0.317)(-0.689) = -0.002088 + 0.292092 + 0.218413 = 0.508417 \approx 0.508$.
- pimples: $(-0.018)(-0.940) + (0.404)(0.601) + (-0.317)(0.146) = 0.016920 + 0.242804 - 0.046282 = 0.213442 \approx 0.213$.
- zebra: $(-0.018)(-0.622) + (0.404)(0.811) + (-0.317)(0.640) = 0.011196 + 0.327644 - 0.202880 = 0.135960 \approx 0.136$.
- idiot: $(-0.018)(-0.077) + (0.404)(-0.375) + (-0.317)(-0.056) = 0.001386 - 0.151500 + 0.017752 = -0.132362 \approx -0.132$.

Step 2 **Forward Pass: Sigmoid Output.** Compute $\sigma(z) = \frac{1}{1+e^{-z}}$:

- $\sigma(0.508) = \frac{1}{1+e^{-0.508}} = 0.6243 \approx 0.624$.
- $\sigma(0.213) = \frac{1}{1+e^{-0.213}} = 0.5531 \approx 0.553$.
- $\sigma(0.136) = \frac{1}{1+e^{-0.136}} = 0.5339 \approx 0.534$.
- $\sigma(-0.132) = \frac{1}{1+e^{0.132}} = 0.4671 \approx 0.467$.

Step 3 **Backward Pass: Prediction Error $e_j = \sigma - t_j$.**

- Stark ($t = 1$): $e_1 = 0.624 - 1.0 = -0.376$.
- pimples ($t = 0$): $e_2 = 0.553 - 0.0 = 0.553$.
- zebra ($t = 0$): $e_3 = 0.534 - 0.0 = 0.534$.
- idiot ($t = 0$): $e_4 = 0.467 - 0.0 = 0.467$.

Step 4 **Gradients for Context Vectors $\frac{\partial L}{\partial \mathbf{c}_j} = e_j \mathbf{w}$.** Multiply error e_j by vector $\mathbf{w} = [-0.018, 0.404, -0.317]$:

- Stark: $-0.376 \times [-0.018, 0.404, -0.317] = [0.00677, -0.15190, 0.11919]$.
- pimples: $0.553 \times [-0.018, 0.404, -0.317] = [-0.00995, 0.22341, -0.17530]$.
- zebra: $0.534 \times [-0.018, 0.404, -0.317] = [-0.00961, 0.21574, -0.16928]$.
- idiot: $0.467 \times [-0.018, 0.404, -0.317] = [-0.00841, 0.18867, -0.14804]$.

Step 5 **Update Context Vectors $\mathbf{c}_j \leftarrow \mathbf{c}_j - \eta \frac{\partial L}{\partial \mathbf{c}_j}$ with $\eta = 0.05$.**

- Stark: $[0.116, 0.723, -0.689] - 0.05 \times [0.00677, -0.15190, 0.11919] = [0.116, 0.731, -0.695]$.
- pimples: $[-0.940, 0.601, 0.146] - 0.05 \times [-0.00995, 0.22341, -0.17530] = [-0.940, 0.590, 0.155]$.
- zebra: $[-0.622, 0.811, 0.640] - 0.05 \times [-0.00961, 0.21574, -0.16928] = [-0.622, 0.800, 0.648]$.

- idiot: $[-0.077, -0.375, -0.056] - 0.05 \times [-0.00841, 0.18867, -0.14804] = [-0.077, -0.384, -0.049]$.

Step 6 **Gradient for Target Vector $\frac{\partial L}{\partial \mathbf{w}} = \sum_{j=1}^{4} e_j \mathbf{c}_j$.** Compute each $e_j \mathbf{c}_j$:

- $-0.376 \times [0.116, 0.723, -0.689] = [-0.04362, -0.27185, 0.25906]$.
- $0.553 \times [-0.940, 0.601, 0.146] = [-0.51982, 0.33235, 0.08074]$.
- $0.534 \times [-0.622, 0.811, 0.640] = [-0.33215, 0.43307, 0.34176]$.
- $0.467 \times [-0.077, -0.375, -0.056] = [-0.03596, -0.17512, -0.02615]$.

Sum all four component-wise: $\frac{\partial L}{\partial \mathbf{w}} = [-0.93155, 0.31845, 0.65541]$.

Step 7 **Update Target Vector $\mathbf{w} \leftarrow \mathbf{w} - 0.05 \frac{\partial L}{\partial \mathbf{w}}$.** $\mathbf{w}_{\text{new}} = [-0.018, 0.404, -0.317] - 0.05 \times [-0.93155, 0.31845, 0.65541]$
$= [-0.018 - (-0.04658), 0.404 - 0.01592, -0.317 - 0.03277]$
$= [\mathbf{0.029}, \mathbf{0.388}, \mathbf{-0.350}]$.

KEY TAKEAWAY

SGNS turns vector learning into simple logistic regression updates. Each step pulls target words toward true neighbors and pushes them away from noise words.

8 Global Ratios, Word Math, and Spam Detection

GloVe captures global corpus statistics; vector arithmetic solves analogies.

Word2Vec scans text within small sliding local windows. In 2014, Pennington, Socher, and Manning created **GloVe** (Global Vectors). GloVe blends local context windows with global matrix counts across the entire library.

8.1 The GloVe Probability Ratio Intuition

Let X denote the matrix of word-word co-occurrence counts. X_{ij} counts how often word j appears in the context of word i. The total context count for word i is $X_i = \sum_k X_{ik}$. The conditional probability of word k given word i is:

$$P_{ik} = P(k \mid i) = \frac{X_{ik}}{X_i} \tag{22}$$

GloVe's key insight is that meaning lives in the **ratio of co-occurrence probabilities**, rather than raw probabilities alone:

$$\text{Ratio} = \frac{P(k \mid \text{ice})}{P(k \mid \text{steam})} \tag{23}$$

Test this ratio on different probe words k:

- For $k =$ "solid": $P(k \mid \text{ice})$ is large, $P(k \mid \text{steam})$ is tiny. The ratio is very large ($\gg 1$).
- For $k =$ "gas": $P(k \mid \text{ice})$ is tiny, $P(k \mid \text{steam})$ is large. The ratio is very small ($\ll 1$).
- For $k =$ "water": Both relate to water. The ratio sits close to 1.

- For $k =$ "fashion": Neither relates to water phases. The ratio sits near 1.

8.2 Word Arithmetic and Analogical Reasoning

Dense embeddings capture regular linear directions in meaning space. We can solve analogy questions ("apple is to tree as grape is to what?") using simple vector addition and subtraction:

$$\mathbf{v}_{\text{vine}} \approx \mathbf{v}_{\text{grape}} - \mathbf{v}_{\text{apple}} + \mathbf{v}_{\text{tree}} \tag{24}$$

The classic example reveals gender direction:

$$\mathbf{v}_{\text{queen}} \approx \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \tag{25}$$

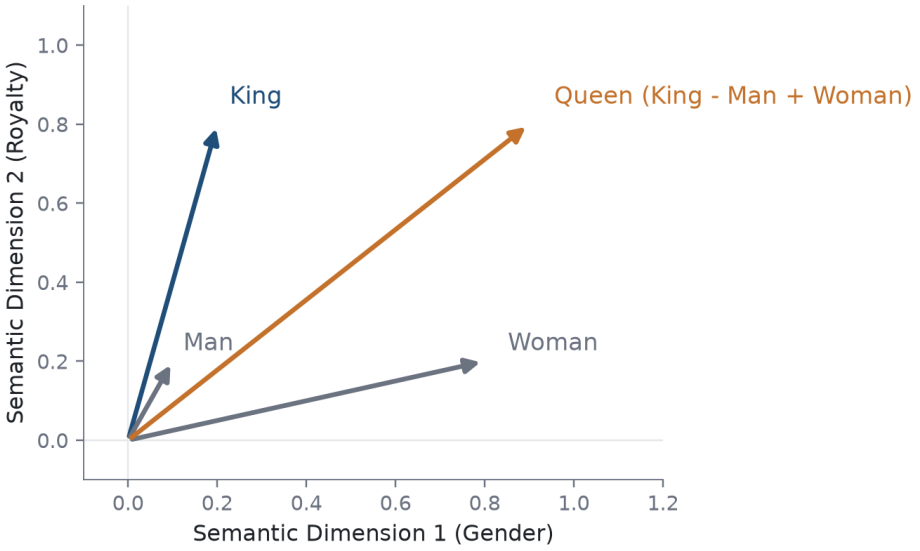


Figure 9. The parallelogram model of analogical reasoning. Shifting by the gender vector transforms king to queen.

8.3 End-to-End Spam Classification Pipeline

In our laboratory experiments, we connect these pieces into a complete text classification system:

1. **CountVectorizer**: Tokenizes text, filters stop-words, and builds bag-of-words count vectors.
2. **TfidfTransformer**: Rescales raw counts into normalized TF-IDF scores.
3. **Multinomial Naive Bayes**: Learns conditional probability $P(\text{word} \mid \text{spam})$ to classify unseen SMS messages.

> **KEY TAKEAWAY**
>
> *GloVe uses global probability ratios to learn strong embeddings. Vector spaces allow linear analogies, and TF-IDF pipelines feed standard machine learning classifiers.*

Self-Test Questions

Test your understanding of these core ideas. Formulate your own answer before reading the upside-down explanation below.

Question 1. Why is an accuracy score of 95% dangerous when evaluating a rare fraud detection system where fraud occurs in 1% of transactions?

A dumb model that labels every single transaction as safe achieves 99% accuracy while missing 100% of fraud cases. In imbalanced problems, accuracy is misleading; you must evaluate precision and recall.

Question 2. In Skip-Gram with Negative Sampling, why do we use $U(w)^{3/4}$ rather than the true unigram distribution $U(w)$ for sampling negative words?

Raising probability to the power 0.75 boosts rare words. It also dampens frequent words like 'the', giving better contrast.

Question 3. Why does raw dot product fail as a reliable similarity measure between word vectors?

Dot product suffers from length bias. Frequent words appear more often and have longer geometric lengths, artificially inflating dot product scores regardless of topical meaning.

Question 4. What is the practical difference between stemming and lemmatization on the word "studies"?

Stemming cuts endings by fixed rules and outputs 'studi' (not a real word). Lemmatization checks a vocabulary dictionary with part of speech tags and outputs the real lemma 'study'.

THE ONE-PAGE POCKET CARD

Core Concepts in One Line:

- **Natural Language**: Fluid, ambiguous, context-dependent human communication.
- **Preprocessing**: Clean $\rightarrow$ Tokenize $\rightarrow$ Stopword Filter $\rightarrow$ Lemmatize.
- **Distributional Hypothesis**: Words occurring in similar contexts share similar meanings.
- **Cosine Similarity**: $\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$, measures angle while ignoring vector length.
- **TF-IDF**: TF $\times$ IDF $= \log_{10}(\text{count}+1) \times \log_{10}(N/df_t)$.
- **Word2Vec SGNS**: $P(+ \mid w, c) = \sigma(\mathbf{w}\cdot\mathbf{c})$, adjusts embeddings with SGD.
- **GloVe**: Embeds words by matching ratios of co-occurrence probabilities.

The Three Essential Equations:

$$\text{Cosine Similarity:} \quad \cos(\theta) = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2}\sqrt{\sum_{i=1}^{d} v_i^2}}$$

$$\text{Harmonic F1-Score:} \quad F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{SGNS Probability:} \quad P(+ \mid w, c) = \frac{1}{1 + e^{-\mathbf{w}\cdot\mathbf{c}}}$$

The Golden Decision Rule:

- Use **Precision** when false alarms carry a heavy penalty (e.g. deleting important emails).
- Use **Recall** when missing a positive case is catastrophic (e.g. medical diagnosis).
- Use **Dense Embeddings** (100–300 dims) over sparse counts to capture semantic generalization.

Glossary & Symbols

Key Terms.

Tokenization Splitting raw continuous text into discrete words or sentences.

Lemmatization Reducing inflected words to valid dictionary headwords using morphological tags.

Precision Fraction of positive model predictions that were correct ($TP/(TP+FP)$).

Recall Fraction of actual positive instances found by the model ($TP/(TP+FN)$).

Cosine Similarity Dot product normalized by vector lengths; evaluates angular alignment.

TF-IDF Weighting scheme balancing term count against document rarity.

Embedding Compact real-valued vector representing semantic meaning in geometric space.

Skip-gram Word2Vec setup where a center word predicts surrounding context words.

Negative Sampling Training trick pairing real context words with random noise words.

GloVe Unsupervised embedding algorithm modeling global word co-occurrence ratios.

Symbols at a Glance.

Symbol	Meaning
TP, FP, FN, TN	True Positives, False Positives, False Negatives, True Negatives
$\mathbf{w}$	Dense target word embedding vector
$\mathbf{c}$	Dense context word embedding vector
$\sigma(z)$	Sigmoid activation function, $1/(1+e^{-z})$, mapping $\mathbb{R} \to (0,1)$
η	Learning rate parameter for gradient descent update steps
N	Total number of documents in a collection
df_t	Document frequency: count of documents containing term t
X_{ij}	Co-occurrence count of word j appearing in the neighborhood of word i
$\|\mathbf{u}\|$	Euclidean L_2 norm (geometric length) of vector $\mathbf{u}$